

DOKUMEN PERENCANAAN TEKNIS

MASTER PLANNING PROYEK VERSION CONTROL UNDANG- UNDANG

Perencanaan implementasi lokal, arsitektur fundamental, alur kerja vibe coding, dan kesiapan kolaborasi dengan ahli hukum

Versi Perencanaan Awal - Pengembangan Lokal

Agustus 2026

Master Planning Proyek Version Control Undang-Undang

Prinsip utamanya:

AI menjadi pelaksana teknis, sedangkan arsitektur, aturan domain, kontrak data, dan standar kualitas harus ditentukan terlebih dahulu olehmu bersama ahli hukum.

Jangan memulai dengan meminta AI “buatkan seluruh platform”. Bangun satu alur kecil secara utuh, valid, dan dapat diuji, kemudian perluas secara bertahap.

1. Target Produk Tahap Pertama

MVP hanya perlu membuktikan alur berikut:

```
Pengguna membuka satu undang-undang
↓
Melihat bagian yang pernah berubah
↓
Mengklik label perubahan
↓
Melihat riwayat perubahan
↓
Melihat peraturan dan ketentuan pengubah
↓
Membandingkan bunyi lama dan baru
↓
Membuka dokumen resmi
```

Cakupan MVP

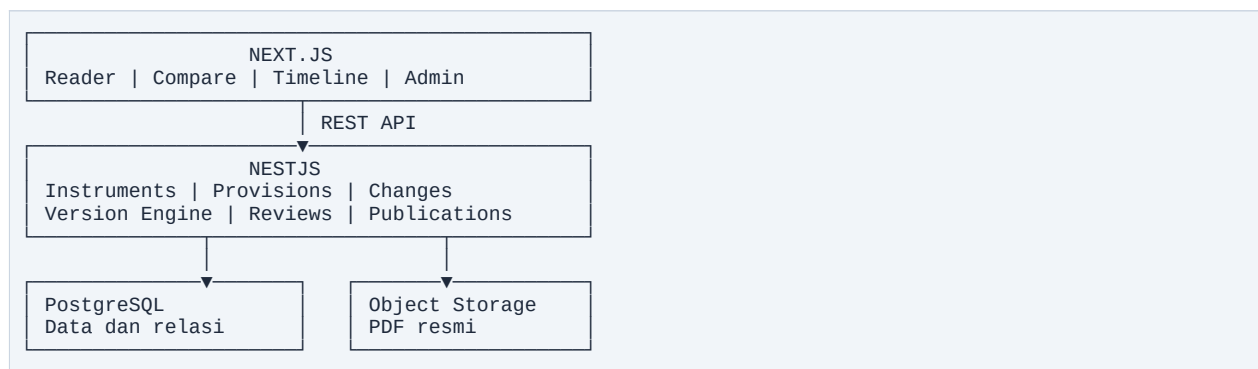
- ✓ Satu keluarga undang-undang
- ✓ Dua atau tiga versi
- ✓ Struktur sampai pasal dan ayat
- ✓ Label perubahan
- ✓ Riwayat perubahan
- ✓ Bunyi sebelum dan sesudah
- ✓ Sumber hukum
- ✓ Proses review ahli hukum
- ✓ Audit log

Belum masuk MVP

- ✗ Seluruh undang-undang Indonesia
- ✗ Scraping massal
- ✗ AI pemberi pendapat hukum
- ✗ Analisis dampak otomatis
- ✗ Chatbot hukum
- ✗ Aplikasi mobile
- ✗ Pembayaran
- ✗ Microservices

2. Arsitektur yang Dikunci

Gunakan **modular monolith** dalam monorepo.



Next.js menggunakan struktur routing berbasis folder, sehingga halaman reader, compare, dan admin dapat dipisahkan secara konsisten. NestJS mengorganisasi aplikasi menggunakan module, controller, dan provider, sehingga modular monolith tetap dapat dikembangkan tanpa langsung beralih ke microservices.

Sumber resmi: [Next.js App Router Documentation](#); [NestJS Modules Documentation](#).

Stack yang digunakan

Komponen	Pilihan
Frontend	Next.js + TypeScript
Backend	NestJS + TypeScript
Database	PostgreSQL
ORM	Prisma
API	REST + OpenAPI
Penyimpanan dokumen	S3-compatible storage
Testing backend	Jest
Testing frontend/E2E	Playwright
Local infrastructure	Docker Compose
Package manager	pnpm
Repository	GitHub private repository

Prisma memberikan akses database yang type-safe dan mencatat perubahan skema melalui migration history. File migrasi SQL tetap dapat disesuaikan ketika dibutuhkan fitur PostgreSQL yang lebih spesifik.

Sumber resmi: [Prisma Migrate Documentation](#).

3. Susunan Local Development

Untuk pengembangan di laptop:

```
Laptop
├─ Next.js berjalan di host
├─ NestJS berjalan di host
├─ PostgreSQL di Docker
├─ Object storage di Docker
└─ Browser membuka localhost
```

Frontend dan backend berjalan langsung di laptop agar hot reload cepat. Database dan penyimpanan file dijalankan melalui Docker Compose supaya lingkungan dapat direplikasi ketika orang lain bergabung.

Docker Compose memungkinkan seluruh service, network, dan volume didefinisikan dalam satu berkas konfigurasi serta dijalankan dengan satu perintah.

Sumber resmi: Docker Compose Documentation.

Target perintah lokal:

```
pnpm install
docker compose up -d
pnpm db:migrate
pnpm db:seed
pnpm dev
```

Tujuan akhirnya:

```
pnpm setup
pnpm dev
```

Anggota baru harus dapat menjalankan proyek tanpa mengatur database secara manual.

4. Struktur Repository

```

legal-version-control/
├── apps/
│   ├── web/           # Next.js
│   └── api/           # NestJS
├── packages/
│   ├── shared-types/  # Tipe data bersama
│   ├── legal-domain/  # Enum dan aturan domain
│   ├── api-client/    # Client API frontend
│   └── ui/            # Komponen UI
├── prisma/
│   ├── schema.prisma
│   ├── migrations/
│   └── seed.ts
├── storage/
│   └── local/         # Hanya development
├── docs/
│   ├── PRODUCT.md
│   ├── DOMAIN_MODEL.md
│   ├── ARCHITECTURE.md
│   ├── DATABASE.md
│   ├── API.md
│   ├── LEGAL_REVIEW.md
│   ├── TESTING.md
│   ├── ROADMAP.md
│   └── adr/
├── tests/
│   ├── fixtures/
│   └── golden/
├── infrastructure/
│   ├── docker-compose.yml
│   └── scripts/
├── .env.example
├── AGENTS.md
├── CONTRIBUTING.md
├── README.md
├── pnpm-workspace.yaml
└── package.json
  
```

5. Empat Lapisan Data

Fondasi database sebaiknya dibagi menjadi empat lapisan.

Lapisan 1 - Sumber resmi

Menyimpan dokumen yang diperoleh dari sumber resmi.

```

SourceDocument
├── URL sumber
├── Institusi penerbit
├── File PDF
├── Hash file
├── Tanggal diperoleh
├── Nomor Lembaran Negara
└── Status pemeriksaan
  
```

Dokumen ini bersifat **immutable**. Jika ditemukan file yang lebih benar, file lama tidak ditimpa. Sistem menambahkan rekaman sumber baru.

Lapisan 2 - Struktur norma

```
LegalInstrument
├── Provision
│   ├── Bab
│   ├── Bagian
│   ├── Paragraf
│   ├── Pasal
│   ├── Ayat
│   ├── Huruf
│   └── Angka
```

Setiap bagian mempunyai identitas stabil:

```
uu-11-2008/pasal-27/ayat-3
```

Identitas tetap sama walaupun isi norma berubah.

Lapisan 3 - Peristiwa perubahan

```
ChangeSet
├── Peraturan pengubah
├── Peraturan target
├── Tanggal mulai berlaku
├── Dasar perubahan
└── ChangeOperation[]
```

Contoh operasi:

```
REPLACE_PROVISION
Target : Pasal 27 ayat (3)
Sumber : Pasal I angka ... UU ...
Lama   : ...
Baru   : ...
```

Lapisan 4 - Versi yang diterbitkan

```
Publication
├── Versi 2008
├── Versi 2016
└── Versi 2024
```

Publication merupakan snapshot siap baca. Snapshot dapat dibuat ulang dari versi awal dan seluruh perubahan yang telah disetujui.

6. Tabel Database Fundamental

Gunakan struktur awal berikut:

```
users
legal_instruments
source_documents
provisions
provision_revisions
change_sets
change_operations
publications
publication_items
legal_relations
review_records
audit_logs
```

Hubungan utamanya

```

LEGAL_INSTRUMENT
├── memiliki SOURCE_DOCUMENT
├── memiliki PROVISION
├── menjadi sumber CHANGE_SET
└── menjadi target CHANGE_SET

PROVISION
├── memiliki PROVISION_REVISION
├── menjadi target CHANGE_OPERATION
└── masuk ke PUBLICATION_ITEM

CHANGE_SET
├── memiliki CHANGE_OPERATION
├── memiliki REVIEW_RECORD
└── menghasilkan PUBLICATION

```

Tanggal yang perlu dibedakan

Jangan hanya memiliki satu kolom tanggal.

```

enacted_at      = tanggal ditetapkan
promulgated_at  = tanggal diundangkan
effective_from   = mulai mempunyai akibat hukum
recorded_at     = dimasukkan ke platform
reviewed_at     = diperiksa reviewer
published_at    = ditampilkan kepada pengguna

```

Pemisahan ini penting karena waktu berlakunya norma berbeda dari waktu platform memperoleh atau memublikasikan data.

7. Operasi Perubahan MVP

Database boleh dirancang untuk banyak jenis perubahan, tetapi mesin pertama hanya perlu mengimplementasikan:

```

ADD_PROVISION
REPLACE_PROVISION
REPEAL_PROVISION

```

Setelah ketiganya stabil, tambahkan:

```

INSERT_BEFORE
INSERT_AFTER
REPLACE_TEXT
DELETE_TEXT
RENUMBER
MOVE
PARTIAL_REPEAL

```

Jangan langsung membuat mesin generik untuk semua kemungkinan. Buat antarmukanya fleksibel, tetapi implementasinya bertahap.

8. Status Workflow Hukum

Gunakan state machine yang jelas:

```

DRAFT
↓
IN_REVIEW
↓
APPROVED
↓
PUBLISHED

```

Apabila ada kesalahan:

```

IN_REVIEW
↓
CHANGES_REQUESTED
↓
DRAFT

```

Aturan penting:

- Data PUBLISHED tidak boleh diedit langsung.
- Koreksi menghasilkan revisi baru.
- Kurator tidak boleh menyetujui pekerjaannya sendiri.
- Setiap keputusan review mempunyai catatan.
- Semua perubahan dicatat dalam audit log.

9. Alur Kerja Data

```

1. Admin memasukkan metadata peraturan
↓
2. Admin menambahkan PDF dan sumber resmi
↓
3. Dokumen dipecah menjadi Bab-Pasal-Ayat
↓
4. Kurator memeriksa struktur hasil pemecahan
↓
5. Kurator membuat Change Set
↓
6. Kurator menambahkan operasi perubahan
↓
7. Sistem membuat preview lama dan baru
↓
8. Reviewer hukum memeriksa
↓
9. Change Set disetujui
↓
10. Version engine membuat publication snapshot
↓
11. Snapshot diterbitkan
↓
12. Mahasiswa membaca dan membandingkan

```

Pembuatan publication harus dilakukan dalam transaksi database. Snapshot, item versi, status perubahan, dan audit log harus berhasil seluruhnya atau dibatalkan seluruhnya. Prisma menyediakan transaction API untuk menjamin serangkaian operasi database berhasil atau gagal sebagai satu kesatuan.

Sumber resmi: *Prisma Client Transactions Documentation*.

10. Modul Backend

```
AuthModule
UsersModule
LegalInstrumentsModule
SourceDocumentsModule
ProvisionsModule
ChangeSetsModule
VersionEngineModule
PublicationsModule
LegalRelationsModule
ReviewWorkflowModule
AuditLogModule
SearchModule
```

Aturan dependensinya:

```
PublicationsModule
  ↓
VersionEngineModule
  ↓
ChangeSetsModule
  ↓
ProvisionsModule
  ↓
LegalInstrumentsModule
```

Hindari hubungan dua arah antar-module. Jika Module A terus memanggil Module B dan Module B memanggil Module A, desain domain perlu diperbaiki.

11. API yang Dibangun

Gunakan OpenAPI sebagai kontrak API. OpenAPI menyediakan deskripsi HTTP API yang dapat dipahami manusia dan mesin tanpa perlu membaca source code backend.

Sumber resmi: OpenAPI Specification.

API publik

```
GET /api/v1/instruments
GET /api/v1/instruments/{id}
GET /api/v1/instruments/{id}/publications
GET /api/v1/publications/{id}/document
GET /api/v1/provisions/{id}/history
GET /api/v1/provisions/{id}/relations
GET /api/v1/compare?from={id}&to={id}
GET /api/v1/search?q={keyword}
```

API internal

```
POST /api/v1/admin/instruments
POST /api/v1/admin/source-documents
POST /api/v1/admin/provisions
POST /api/v1/admin/change-sets
POST /api/v1/admin/change-sets/{id}/operations
POST /api/v1/admin/change-sets/{id}/submit
POST /api/v1/reviews/{id}/approve
POST /api/v1/reviews/{id}/request-changes
POST /api/v1/publications
```

Format respons API

Selalu konsisten:

```
{
  "data": {},
  "meta": {
    "requestId": "req_123"
  },
  "error": null
}
```

Kesalahan:

```
{
  "data": null,
  "meta": {
    "requestId": "req_123"
  },
  "error": {
    "code": "CHANGE_SET_NOT_APPROVED",
    "message": "Change set harus disetujui sebelum dipublikasikan."
  }
}
```

12. Strategi Vibe Coding

Aturan utama

Satu issue, satu tujuan, satu perubahan kecil, satu rangkaian pengujian.

Jangan memberikan prompt:

"Buatkan backend version control undang-undang secara lengkap."

Gunakan prompt seperti:

```
Tujuan:
Tambahkan endpoint GET /api/v1/provisions/:id/history.

Konteks:
Provision memiliki banyak provision revisions dan change operations.

File yang boleh diubah:
- apps/api/src/provisions/*
- packages/shared-types/*
- apps/api/test/provisions/*

Di luar cakupan:
- Jangan mengubah schema database.
- Jangan mengubah authentication.
- Jangan melakukan refactor module lain.

Kriteria penerimaan:
1. Mengembalikan seluruh revisi secara kronologis.
2. Menampilkan sumber perubahan.
3. Mengembalikan 404 jika provision tidak ditemukan.
4. Memiliki unit dan integration test.
5. OpenAPI diperbarui.

Sebelum mengimplementasikan:
Jelaskan rencana perubahan dan file yang akan disentuh.
```

Siklus kerja setiap fitur

1. Buat issue
2. Tulis acceptance criteria
3. Minta AI membuat rencana
4. Periksa rencana
5. Minta implementasi
6. Periksa diff
7. Jalankan lint
8. Jalankan typecheck
9. Jalankan test
10. Uji manual
11. Commit
12. Tutup issue

Perintah pemeriksaan minimum:

```
pnpm lint
pnpm typecheck
pnpm test
pnpm test:integration
pnpm test:e2e
```

13. File Konteks untuk AI

Buat AGENTS.md di root repository. AI harus membaca file tersebut sebelum mengubah kode.

Contoh isinya:

```
# Project Rules

1. Gunakan TypeScript strict mode.
2. Jangan mengakses database dari controller.
3. Semua business rule berada pada service/domain layer.
4. Jangan mengubah published data secara langsung.
5. Semua perubahan schema wajib menggunakan migration.
6. Jangan menambahkan dependency tanpa persetujuan.
7. Setiap endpoint harus mempunyai DTO dan OpenAPI.
8. Setiap fitur harus mempunyai test.
9. Jangan menghapus audit log.
10. Jangan membuat kesimpulan hukum otomatis.
```

Tambahkan dokumen:

```
docs/PRODUCT.md
docs/DOMAIN_MODEL.md
docs/ARCHITECTURE.md
docs/DATABASE.md
docs/API.md
docs/LEGAL_REVIEW.md
docs/TESTING.md
```

AI tidak boleh menjadi sumber kebenaran arsitektur. Dokumen-dokumen tersebut yang menjadi sumber kebenaran.

14. Definition of Ready

Sebuah fitur baru boleh dikerjakan apabila:

- ✓ Tujuan pengguna jelas
- ✓ Skenario hukumnya jelas
- ✓ Sumber data tersedia
- ✓ Input dan output diketahui
- ✓ Acceptance criteria tertulis
- ✓ File atau module target diketahui
- ✓ Tidak bertentangan dengan keputusan arsitektur

Contohnya, fitur “riwayat pasal” belum siap dikerjakan jika tim hukum belum memutuskan apakah putusan MK masuk sebagai perubahan versi atau catatan pengaruh yudisial.

15. Definition of Done

Sebuah fitur dianggap selesai apabila:

- ✓ Acceptance criteria terpenuhi
- ✓ TypeScript tidak menghasilkan error
- ✓ Lint berhasil
- ✓ Unit test berhasil
- ✓ Integration test berhasil
- ✓ E2E test yang relevan berhasil
- ✓ API documentation diperbarui
- ✓ Migration dan seed dapat dijalankan
- ✓ Tidak ada secret dalam source code
- ✓ Audit log tersedia untuk operasi penting
- ✓ README diperbarui bila alur penggunaan berubah
- ✓ Uji manual telah dilakukan

Playwright cocok untuk pengujian alur end-to-end seperti membuka undang-undang, mengklik label, melihat riwayat, dan membandingkan versi. Playwright Test dapat menjalankan satu test, kelompok test, atau keseluruhan test suite.

Sumber resmi: *Playwright Test Documentation*.

16. Strategi Pengujian

Unit test

Menguji aturan kecil:

- Mengurutkan perubahan berdasarkan `effective_from`
- Menentukan versi aktif pada tanggal tertentu
- Menolak publication dari change set yang belum disetujui
- Menolak perubahan target yang tidak ditemukan

Golden test

Ini sangat penting untuk version engine.

Simpan contoh input:

```
tests/fixtures/sample-law/  
├─ original.json  
├─ amendment-1.json  
└─ amendment-2.json
```

Simpan hasil yang diharapkan:

```
tests/golden/sample-law/  
├─ version-1.json  
├─ version-2.json  
└─ version-3.json
```

Setiap perubahan kode version engine harus menghasilkan output yang sama. Jika berubah, developer wajib menjelaskan alasannya.

Integration test

- Membuat instrument
- Menambahkan provision
- Membuat change set
- Mengajukan review
- Menyetujui review
- Memublikasikan versi

E2E test

- Pengguna membuka reader
 - Memilih versi
 - Mengklik label perubahan
 - Membuka sumber
 - Membandingkan dua versi
-

17. Roadmap Implementasi

Fase 0 - Mengunci domain

Durasi: 3-5 hari

Hasil:

```
PRODUCT.md
DOMAIN_MODEL.md
Daftar istilah hukum
Daftar jenis perubahan
Aturan review
Pemilihan undang-undang pilot
10-15 contoh kasus perubahan
```

Belum ada coding fitur.

Selesai ketika: ahli hukum memahami dan menyetujui model konseptual awal.

Fase 1 - Fondasi repository dan local development

Durasi: 2-4 hari

Hasil:

```
Monorepo
Next.js
NestJS
PostgreSQL
Prisma
Docker Compose
Environment variables
Migration pertama
Seed data
Lint, formatting, dan test command
```

Selesai ketika: proyek dapat dijalankan ulang dari laptop kosong hanya melalui dokumentasi README.

Fase 2 - Vertical slice pertama

Durasi: 4-7 hari

Gunakan dokumen hukum buatan terlebih dahulu, bukan undang-undang nyata.

Contoh:

```
Peraturan Contoh Nomor 1
Pasal 1 versi awal
Pasal 1 diubah oleh Peraturan Contoh Nomor 2
```

Bangun alur lengkap:

```
Database
↓
API
↓
Reader
↓
Label perubahan
↓
Panel riwayat
↓
Compare
```

Selesai ketika: satu perubahan dapat dilihat dari frontend sampai sumber perubahan.

Vertical slice ini membuktikan arsitektur tanpa tersandera kompleksitas hukum.

Fase 3 - Version engine

Durasi: 5-10 hari

Implementasikan:

```
ADD_PROVISION
REPLACE_PROVISION
REPEAL_PROVISION
```

Tambahkan:

```
Change set
Change operation
Publication
Provision revision
Golden tests
Database transaction
```

Selesai ketika: versi dapat direkonstruksi secara deterministik dari sumber dan change operations.

Fase 4 - Dashboard kurator dan reviewer

Durasi: 5-8 hari

Fitur:

```
Input dokumen
Input struktur
Pemetaan perubahan
Preview hasil
Submit review
Approve
Request changes
Publish
Audit log
```

Selesai ketika: perubahan tidak perlu dimasukkan langsung melalui database.

Fase 5 - Implementasi undang-undang pilot

Durasi: bergantung pada kompleksitas dokumen

Alur:

```

Kumpulkan dokumen resmi
↓
Masukkan struktur
↓
Petakan perubahan
↓
Review bersama ahli hukum
↓
Perbaiki
↓
Publikasikan versi lokal

```

Selesai ketika:

- ✓ Setiap label mempunyai sumber resmi
- ✓ Setiap perubahan mempunyai ketentuan pengubah
- ✓ Versi lama dan baru dapat dibandingkan
- ✓ Reviewer hukum menyetujui hasil
- ✓ Tidak ada data tidak terverifikasi yang tampil sebagai fakta

Fase 6 - Demo kerja sama

Hasil yang dibawa:

1. Demo lokal yang berfungsi
2. Diagram cara kerja
3. Daftar fitur MVP
4. Contoh satu keluarga undang-undang
5. Dashboard review
6. Daftar kebutuhan dari ahli hukum
7. Roadmap pengembangan
8. Pembagian tanggung jawab

18. Pola Kerja Git

Walaupun masih bekerja sendiri, gunakan GitHub dan pull request sejak awal.

```

main
├─ feature/LVC-001-instrument-list
├─ feature/LVC-002-provision-history
└─ fix/LVC-003-version-order

```

Alurnya:

```

Issue
↓
Feature branch
↓
Implementasi
↓
Pull request
↓
Review diff
↓
Test
↓
Merge ke main

```

Pull request menyediakan tempat untuk membahas dan meninjau perubahan sebelum masuk ke branch utama. Ketika kolaborator sudah bergabung, branch protection dapat mewajibkan review dan status check sebelum merge.

Sumber resmi: [GitHub Pull Requests Documentation](#).

Untuk fase lokal:

- main harus selalu bisa dijalankan.
- Jangan coding langsung di main.
- Satu branch untuk satu issue.
- Jangan gabungkan perubahan database dan redesign besar dalam satu PR.
- Commit sebelum meminta AI melakukan refactor lanjutan.

19. Dokumen yang Dibawa kepada Orang Hukum

Kamu tidak perlu membawa source code sebagai bahan utama. Bawa:

Product brief

Menjelaskan:

Masalah
Target pengguna
Solusi
Batas MVP
Cara platform bekerja

Diagram alur

```

Sumber resmi
↓
Struktur norma
↓
Pemetaan perubahan
↓
Review hukum
↓
Publication
↓
Mahasiswa
  
```

Domain glossary

Contoh istilah:

Peraturan awal
Peraturan pengubah
Ketentuan pengubah
Ketentuan target
Pencabutan
Penyisipan
Penggantian
Tanggal berlaku
Naskah konsolidasi
Putusan yang memengaruhi norma

Template pemetaan perubahan

Target	Tindakan	Sumber	Ketentuan sumber	Berlaku	Catatan
Pasal X	Diganti	UU A	Pasal I angka 2	Tanggal	-
Pasal Y	Ditambah	UU B	Pasal I angka 3	Tanggal	Pasal baru

Daftar keputusan yang membutuhkan ahli hukum

- Apa yang dianggap satu perubahan?
- Bagaimana perubahan frasa dicatat?
- Bagaimana putusan MK ditampilkan?
- Apakah penjelasan ikut dibuatkan versi?
- Bagaimana pencabutan sebagian ditampilkan?
- Apa perbedaan dampak langsung dan hubungan terkait?
- Kapan data dianggap layak dipublikasikan?

20. Pembagian Tanggung Jawab

Area	Kamu	Ahli hukum	Mahasiswa
Arsitektur teknologi	Utama	Konsultasi	-
Model perubahan hukum	Implementasi	Utama	Masukan
Validasi sumber	Dukungan	Utama	-
UI dan pengalaman pengguna	Utama	Masukan	Pengujian
Version engine	Utama	Validasi hasil	-
Review data	Sistem	Utama	-
Pengujian kegunaan	Dukungan	Dukungan	Utama

Kebutuhan awal dari ahli hukum sebaiknya dibatasi:

1. Membantu memilih satu UU pilot.
2. Memvalidasi glosarium.
3. Memetakan 10-20 perubahan contoh.
4. Memeriksa hasil prototype.
5. Membantu menentukan standar publikasi.

Jangan meminta mereka mengisi seluruh database.

21. Risiko Utama

Risiko	Pencegahan
Model hukum keliru	Validasi domain sebelum coding
AI mengubah fondasi sembarangan	AGENTS.md dan scope setiap prompt
Database terlalu generik	Mulai dari tiga operasi perubahan
Data resmi berbeda antarportal	Simpan sumber, hash, dan provenance
Hasil parser salah	Human review wajib
Published data tertimpa	Immutable publication

Risiko	Pencegahan
Version engine sulit dipercaya	Golden tests
Proyek terlalu besar	Satu vertical slice dan satu UU pilot
Sulit menerima kolaborator	Docker, seed, README, migration
AI menghasilkan kode tidak konsisten	Lint, typecheck, test, review diff

22. Keputusan yang Harus Dikunci Sekarang

```

Arsitektur      : Modular monolith
Repository      : Monorepo
Frontend        : Next.js
Backend         : NestJS
Database        : PostgreSQL
ORM             : Prisma
Local infra     : Docker Compose
API             : REST + OpenAPI
Model perubahan : Change set + operations
Model pembacaan : Immutable publication snapshots
Validasi        : Human legal review
Pilot           : Satu keluarga undang-undang
Metode coding   : Issue-driven vibe coding

```

Urutan Mulai Besok

```

Hari 1
Buat repository dan folder docs.

Hari 2
Tulis PRODUCT.md, DOMAIN_MODEL.md, dan batas MVP.

Hari 3
Buat 10 contoh skenario perubahan bersama calon ahli hukum.

Hari 4
Buat monorepo, Docker Compose, PostgreSQL, dan Prisma.

Hari 5
Buat schema database awal dan seed dokumen contoh.

Hari 6-8
Bangun vertical slice: reader → label → history → source.

Hari 9-12
Bangun version engine dengan tiga operasi pertama.

Hari 13-16
Bangun review dan publication workflow.

Setelah itu
Masukkan undang-undang pilot bersama ahli hukum.

```

Fondasi terpentingnya bukan banyaknya fitur, tetapi jaminan berikut:

Setiap bunyi norma dapat ditelusuri ke versi, peristiwa perubahan, ketentuan pengubah, dokumen resmi, serta pihak yang memverifikasinya.